StudyBuddy - Group 10 - Iteration 2 Log

Members:

- Sean Lee-Wah - 221344163 - sean1010@my.yorku.ca
- Bhavya Trivedi - 219743590 - Bhavya23@my.yorku.ca
- Aqsa Malik - 221100987 - xsa@my.yorku.ca
- Vaughn - 219846914 - vc8@my.yorku.ca
- Omar Fakousa - 220609426- omar821@my.yorku.ca
- Yash Bhupta - 219743285 - bhupta01@my.yorku.ca

External Links:

Jira: https://le-eecs-2311-project.atlassian.net/jira/software/projects/ID/boards/1

Architecture Diagram (Updated):
https://github.com/hvpham-yorku/project-group-10_StudyBuddy/blob/itr2-finishing-touches/Documents/ITR2/architecture-itr2.png

UML Class Diagrams (Updated):
https://github.com/hvpham-yorku/project-group-10_StudyBuddy/tree/itr2-finishing-touches/Documents/ITR2/ITR2-Class-UML-Diagrams

Meeting Summary Video: Customer meeting's summary video.mp4

Final Working Branch for Iteration 2:
hvpham-yorku/project-group-10_StudyBuddy at itr2-finishing-touches

---

Meeting Minutes:

February 19, 2026 (81 min)

Attendees: Bhavya Trivedi, Vaughn Chan, Yash Bhupta, Omar, Sean, Aqsa

Summary

We held a planning session for the current iteration of StudyBuddy, focusing on defining our revised architecture, establishing our Firebase database collections, and resetting our workflow for a fresh start. We clarified the Firebase authentication process, emphasizing token verification on the backend, and agreed on a strict branching strategy. To maintain momentum and quickly address blockers, we are transitioning from long weekly meetings to 15-minute check-ins every two days.

Key Decisions

- **Workflow:** Implementing a six-branch strategy named by owner to clearly track progress, and shifting to 15-minute stand-ups every two days instead of longer weekly meetings.
- **Database:** Structuring Firebase collections by data type (user tokens, auth, events) and establishing a "stop database" alongside the persistent one. Shared JSON access will be provided to the whole team to reduce individual setup friction.
- **Authentication:** Frontend will manage email/password logins and ID tokens; the backend will verify both the email and token strictly during sign-up to create the user object. JWT tokens will authenticate subsequent API requests.
- **Development Scope:** Prioritizing MVP features (account creation, event management, user interactions) with a strong emphasis on unit testing for this iteration.

Task Assignments

- **Yash:**
  - Implement Firebase authentication, including @yorku.ca/@my.yorku.ca domain-restricted sign-up, email verification, and base profile creation.
- **Vaughn:**
  - Develop event management features, allowing users to create, host, join, and cancel study events or their own attendance.
- **Bhavya:**
  - Build chat and networking features, including post-session connection requests, group messaging with attachments, and typing indicators. *(Due: 2026-03-02)*
  - Define collections and structure for the Firebase database.
  - Ask the professor about tagging commits.
- **Sean:**
  - Create profile management settings, enabling users to update enrolled courses, customize their "Study Vibe", and manage privacy visibility.
- **Omar:**
  - Build connection tracking, including a dedicated accepted connections page and real-time online/idle/offline status indicators for group members.
- **Aqsa:**
  - Implement campus map functionalities to display the user's current location, nearby study sessions with details, and directional navigation.
- **Team / Administrative:**
  - Confirm access to Firebase for all team members and share the setup JSON file *(Assignees: Bhavya, Vaughn)*.

    - Assign User Flows and share the updated timeline document with the team.
    - Define the "stop database" structure and conduct unit testing.
    - Integrate and prepare the MVP demo for the end of the iteration.

February 28, 2026 (40 min)

Attendees: Bhavya Trivedi, Omar, Sean, Aqsa Malik

Summary

We held a check-in meeting to review our progress on Iteration 2 features, troubleshoot technical blockers, and prepare for our upcoming submission. This was to complement our regular chat progress updates which happen every other day. We confirmed that the Firebase database is online and accessible to the team. Development updates focused on map integration challenges, chat database connections, and user profile data retrieval. We also emphasized the requirement for unit tests and began planning our project presentation, scheduling a dedicated working session for Sunday.

Key Decisions

- **Database & Architecture:** Firebase is online; user profile data will be stored in Firestore and queried via user IDs. Architecture documentation has been updated to reflect these data flows.
- **Feature Logic:** Direct messaging functionality is actively being debugged. Group chats will strictly enforce the rule requiring users to have completed a study session before gaining access.
- **Development Standards:** The team agreed to prioritize regular, incremental commits to avoid integration issues and mandated that unit tests must be written for the current iteration.
- **Presentation Planning:** We will pivot some focus toward the e-class submission, with a dedicated presentation prep meeting scheduled for tomorrow (Sunday).

Task Assignments

- **Bhavya:**
    - Follow up on Firebase database access to ensure the whole team is unblocked.
    - Finalize chat functionality, including fixing the DM bug and enforcing the completed-session rule for group chats.
- **Omar & Sean:**
    - Coordinate with Yash on integrating frontend authentication features and profile page data retrieval.
- **Aqsa:**
    - Continue map implementation, specifically tackling Google Maps integration for moving and pin-dropping.

- **Yash:**
    - Update the authentication code to align with the new Firestore structure.
- **Vaughn:**
    - Continue assigned detailed user stories regarding events functionality.
- **Team:**
    - Write required unit tests for all assigned features.
    - Practice and prepare for the final Deliverable 1 Submission.

March 4, 2026 (75 min)

Attendees: Bhavya Trivedi, Vaughn Chan, Sean Lee-Wah, Aqsa Malik, Yash Bhupta, Omar Medhat Fakoussa

Summary

We focused on finalizing code integration and preparing comprehensive documentation for our upcoming submission. A major technical hurdle discussed was implementing a local "stub" database as a fallback in case Firebase goes down, which requires updates to our Spring application properties. We also reviewed the status of the chat integration—specifically ensuring that user connection requests function correctly before chat sessions can spawn. With the deadline approaching, we emphasized a strict requirement for comprehensive unit and integration testing across all branches before merging, and scheduled a final documentation meeting for Thursday at 8:30 PM.

Key Decisions

- **Database Architecture:** We will implement a local stub database to serve as a persistent fallback if Firestore is unavailable. The switch between the real database and the stub will be managed via Spring application properties.
- **Chat Feature Logic:** Chat session management is tied to core user connections; specific group chats will actively block access based on user identity, meaning connection requests must work perfectly first.
- **Testing Standards:** All merged code must have corresponding unit and integration tests covering edge cases. Tests must be consolidated into a designated, separate test folder.
- **Documentation:** The Architecture Document, wiki, and installation scripts must be updated to clearly distinguish between Iteration 1 and Iteration 2 features.

Task Assignments

- **Aqsa:**
    - Implement local database logic and finalize database integration (with Sean).
    - Update the Architecture Wiki with recent changes.
- **Sean:**
    - Implement local database logic (with Aqsa).
    - Create and finalize documentation (with Yash).
    - Handle the final merge of code and tests.

- **Vaughn:**
  - Coordinate code changes for compatibility (with Yash).
  - Continue examining branches and managing the merge process.
- **Yash:**
  - Coordinate code changes for compatibility (with Vaughn).
  - Create and finalize overall project documentation (with Sean).
- **Omar:**
  - Complete assigned unit tests.
  - Finalize architecture changes (with Bhavya).
- **Bhavya:**
  - Finalize architecture changes (with team).
- **Team / Shared Tasks:**
  - **Update Architecture Document:** Bhavya, Sean, Yash, Omar *(Due: March 5)*.
  - **AI Usage Declaration:** Prepare the standard disclosure for the submission.
  - **Testing:** Add comprehensive unit and integration tests for all individual components *(Due: March 5)*.
  - **Scripts:** Update installation scripts and README documentation *(Due: March 5)*.
  - **Meeting:** Attend the documentation check-in on Thursday at 8:30 PM.

Rationale of Changes on Plan and Big Design Decisions

Category	ITR1 (Before)	ITR2 (After)
Database	Stub / In-Memory (ITR1)	Firestore (Firebase) — live, persistent
Architecture	MVC pattern, basic layering	Controller → Service → DAO layered architecture with DTOs
Auth	Basic sign-in/sign-up UI, Firebase Auth stub	Full domain-restricted auth (@yorku.ca), email verification, password reset
Chat System	Not implemented	Direct chat, event chat, paginated messages, friend requests

File Uploads	Not implemented	Upload/download attachments via UploadService
Events	Create, host, cancel events	Create, list, delete — plus event-linked chat support
API Surface	Limited / stub endpoints	Full REST API with documented error contract (401/403/404/422/500)
User Profile	Profile page, courses, session log	Extended Student model (study vibes, privacy settings, notifications, 2FA)
Hosting	DigitalOcean (decided in ITR1)	DigitalOcean (continued), backend fully deployed
Testing	Unit test files created	Unit tests required for all features; carried-over auth tests completed

1. Database: Stub/In-Memory → Firestore (Persistent)

- **ITR1 State:** ITR1 used a stub/in-memory database for all data storage. While Firebase was selected as the intended database during the ITR1 planning (February 5 meeting), we deferred the Firestore integration.
- **Change in ITR2:** ITR2 fully integrates Firestore as the live database backend. All DAOs now perform real Firestore read/write operations (create, save, findById, query). The ChatDAO, MessageDAO, and FriendRequestDAO are newly implemented against Firestore.
- **Rationale:** We made the switch from stub to persistent storage to support real-time features and save data across sessions. Firestore's document model fits well to Studybuddy's data schema (Chat, Message, Student, etc), and its real-time listeners will enable future features like typing indicators.

<u>Rationale of why using Firestore is better than SQL for our use case:</u>

Data Durability: Persistent storage ensures that user data (events, chats, friend requests) is saved across sessions and survives server restarts, unlike in-memory/stub DBs which lose all data when the app stops.

Scalability & Real-Time Features: Firestore natively supports real-time updates and scalable document storage, enabling features like live chat, typing indicators, and instant event updates. This is critical for collaborative study sessions in our project.

Flexible Schema: Firestore's document-based model aligns with StudyBuddy's entities (Chat, Message, Student, Event), allowing easy evolution of data structures without rigid SQL table migrations.

Cross-Platform & Cloud Integration: Firestore is cloud-hosted, making it accessible from web, mobile, and backend services, and simplifies authentication and security via Firebase Auth.

Reduced DevOps Overhead: No need to manage SQL servers, backups, or migrations; Firestore handles scaling, replication, and reliability.

Future-Proofing: Real-time listeners and cloud functions in Firestore enable advanced features (notifications, analytics) with minimal backend changes.

2. Architecture: Basic MVC → Layered Controller/Service/DAO with DTOs

- **ITR1 State:** ITR1 established the MVC pattern as a guiding principle. The codebase had Controllers and some Services, but layering was not consistently enforced and DTOs were minimal.
- **Change in ITR2:** ITR2 formalizes a strict four-layer architecture: Frontend sends JSON to Controllers, Controllers map to DTOs, Services handle business logic, and DAOs handle all Firestore operations. A full set of DTOs (MessageResponseDTO, PagedMessagesResponse, FileAttachmentDTO, ChatInboxDTO, etc.) are implemented.
- **Rationale:** Enforcing strict layer separation reduces coupling, improves testability, and ensures that business logic is never mixed into data access code. As the team size and feature count grows, this prevents merge conflicts and makes it easier to assign specific layers to specific team members. The DTO layer also ensures the API contract is explicit and stable, making frontend/backend integrationless buggy.

3. Authentication: Basic UI Forms → Full Auth with Email Verification

- **ITR1 State:** ITR1 completed the sign-in and sign-up UI forms (user stories 4.1 and 4.2) and began Firebase Auth infrastructure (4.3, marked In Progress). Email verification and YorkU domain restriction were planned but not fully wired.
- **Change in ITR2:** ITR2 completes the auth flow: registerUser() restricts sign-up to @yorku.ca and @my.yorku.ca domains, sends a verification email, and stores the Student profile in Firestore. loginUser() blocks unverified accounts. generateResetLink() supports password reset.
- **Rationale:** Completing authentication was a carried-over priority from ITR1 for us. The YorkU domain restriction directly addresses our client's (Wmmsi's) safety requirement identified in ITR1. Email verification prevents fake accounts and ensures that only real YorkU students can access the platform, which is central to the product's trust model.

4. New Feature: Chat System

- **ITR1 State:** No chat system existed in ITR1. Communication between users was identified as a planned feature (User Story 3: Communication) but was in the backlog.
- **Change in ITR2:** ITR2 implements a complete chat system: direct chats between users, event-linked group chats, paginated message history with cursor support, support for TEXT/LINK/FILE message types, and a friend request system gated by shared session attendance.
- **Rationale:** Chat was the highest-priority new feature for ITR2 as it directly enables the core value proposition of StudyBuddy, connecting students. Tying friend requests to shared session attendance reinforces the platform's safety model and encourages real in-person study activity. Paginated message retrieval was added proactively to support performance at scale.

5. New Feature: File Upload/Download

- **ITR1 State:** File sharing was not implemented in ITR1.
- **Change in ITR2:** ITR2 adds an UploadService and DownloadedFile support. Users can upload attachments (validated by type/size), and the system stores files locally with metadata returned via FileAttachmentDTO. FILE-type messages in chat reference these uploads.
- **Rationale:** File sharing is a natural extension of the chat system. Study groups frequently exchange notes, PDFs, and images. Implementing it alongside chat in ITR2 avoids needing to refactor the message model in a later iteration. Local filesystem storage was chosen for simplicity in ITR2, with cloud storage (e.g., Firebase Storage) available as a future upgrade path.

6. Extended Student Model

- **ITR1 State:** The ITR1 Student model covered core fields: userId, email, fullName, program, bio, avatar, courses, attendedEventIds, and year.
- **Change in ITR2:** ITR2 expands the Student model with studyVibes (List<String>), privacySettings (Map<String, Boolean>), notifications (Map<String, Boolean>), twoFAEnabled, autoTimeout, isOnline, joinedDate, and location. Corresponding API endpoints are added (e.g., PUT /{studentID}/study-vibes, PUT /{studentID}/privacy-settings).
- **Rationale:** The expanded model supports user stories deferred from ITR1 around profile customization and privacy (User Stories 1.6 Edit Study Preferences, 2.4 Privacy Policy). Adding these fields now avoids costly Firestore schema migrations later. The privacy settings map gives users granular control over what is visible, which was highlighted as a requirement in early customer discussions.

7. Formalized API Error Contract

- **ITR1 State:** ITR1 did not have a documented API error contract. Error handling was ad hoc across endpoints.
- **Change in ITR2:** ITR2 defines a formal error contract: 401 (unauthenticated), 403 (forbidden), 404 (not found), 413 (payload too large), 422 (validation failure), 500

(server error). This is consistently applied across all Chat, Upload, Event, and Auth endpoints.

- **Rationale:** A documented error contract is essential for reliable frontend integration. Without it, the React frontend cannot reliably distinguish between authentication failures, permission errors, and validation problems, leading to poor user-facing error messages. Standardizing this in ITR2 reduces the number of integration bugs and speeds up frontend development.

Concern with the Project or Group Members

No concerns. We were able to manage our time and stay on top of deliverable much better this iteration.

Task Assignments and Estimated/Actual Time per Development:

User Story	Assignee(s)	Estimated Time	Actual Time	Status	Notes
(1.1) Create the base profile including name, email, and major	Yash	2 days	2 days	Completed	User profile page that user can edit
(4.4) Domain Filter: Restricting sign-ups to only @my.yorku.ca or @yorku.ca emails	Yash	3 days	4 days	Completed	Only allow specific york emails to create profiles
(4.2) Email authentication to ensure secure logins	Yash	3 days	2 days	Completed	Send verification through email for security purposes
(2.1, 2.2) I should be able to select a	Vaughn	3 days	4 days	Completed	Allow users to create event and be

place to create an event and be the host of it					the only one that can make changes to it
(2.4) I should be able to join an existing and upcoming events	Vaughn	3 days	2 days	Completed	Allow users to join events that are currently occurring or will occur in the future
(2.3) Cancel Events: Allow the host to cancel an event they created	Vaughn	1 day	1 day	Completed	Allow only users who created an event, to delete their event
(2.5) Allow the user to cancel their own attendance to an event they joined	Vaughn	2 days	3 days	Completed	Allow users to leave any event they have joined
(1.4) I should be able to send friend requests to students after successfully completing a session	Bhavya	3 days	3 days	Completed	Allow users to send friend requests to other users
(3.3) I should be able to use a chat feature to discuss study plans with group members	Bhavya	5 days	6 days	Completed	Allow users to chat with other users (1 on 1 chat, event chat)
(3.4) I should be able to attach files and links within the group chat	Bhavya	2 days	3 days	Completed	Allow users to include files and links in chat
(3.9) Indicator fo Message: Viewing an indicator when someone is actively typing in the chat	Bhavya	2 days	2 days	Completed	Have a toggle that is visible to everyone in chat when someone is typing
(1.3) Add, remove, or update currently enrolled courses	Sean	2 days	3 days	Completed	Allow users to change their courses
(1.5) Customize “Study Vibe” and	Sean	2 days	2 days	Completed	Allow users to change their study

behavioral preferences					preferences
(4.1) Use explicit settings to reveal or hide certain information about the student	Sean	3 days	4 days	Completed	Allow users to hide certain information when other view their profile
(1.6) Accessing a specific page that displays a list of all accepted connections	Omar	2 days	3 days	Completed	Allow users to view all of their current connections
(3.8) View whether study group members are online, idle, or offline	Omar	2 days	1 day	Completed	Allow users to see current status of other members
(3.1) View available study sessions on campus map near your location	Aqsa	4 days	4 days	Completed	Allow users to see what events are occurring near them through pin drops on map
(3.2) Check session details (course, time, duration) from the map before deciding to join	Aqsa	2 days	3 days	Completed	Allow users to see details of event after clicking on pin drop on map
(3.6) Display your current location on the map	Aqsa	2 days	1 day	Completed	Allow users to see their current location on the given map
(3.7) Get directional navigation from your location to the chosen study spot	Aqsa	2 days	3 days	Completed	Opens a pop-up in a new tab that shows navigation from users current location to location of study spot

Authentication & Onboarding

Description

"I should be able to sign up for an account and sign in using that account"

These tasks corresponds with the following detailed user stories:

Detailed User Story	Description
(1.1) Create User Profile	Display name, email, major, bio, courses, study preferences, and past event attendance.
(4.4) Domain Filter	Restrict sign-ups to email addresses ending in @my.yorku.ca or @yorku.ca.
(4.2) 2FA Authentication	Email-based two-factor authentication for secure logins.

In Progress

Details

Assignee	YB Yash Bhupta Assign to me
Priority	= Medium
Parent	None
Due date	None
Labels	None
Team	None
Start date	None

Event Management

Description

"I should be able to add/remove events that are tied to my account"

These tasks corresponds with the following detailed user stories:

Detailed User Story	Description
(2.1) Create Events	Ability to create an event profile for designated hosting.
(2.2) Host Events	Selection of a specific place to host the study event.
(2.4) Join Existing Events and Upcoming Events	Ability for users to join one or more events per day.
(2.3) Cancel Events	Only the host of the event has the authority to cancel it
(2.5) Cancel Events Attendance	Users can cancel their own attendance for a joined event.

In Progress

Details

Assignee	VC Vaughn Chan Assign to me
Priority	= Medium
Parent	None
Due date	None
Labels	None
Team	None
Start date	None
Sprint	ITR2

Chat & Communication

Description

"I should be able to chat with users and send connection requests"

These tasks corresponds with the following detailed user stories:

Detailed User Story	Description
(1.4) Send Connection Request	Enable friend requests to students after successful sessions.
(3.3) Chat with Other Students	In-app chat feature for group members to discuss study plans.
(3.4, Optional) Ability to attach files and links within the group chat	Ability to attach files and links within the group chat.
(3.9, Optional) Indicator of Message	A typing indicator to facilitate smoother communication

In Progress

Details

Assignee	BT Bhavya Trivedi Assign to me
Priority	= Medium
Parent	None
Due date	None
Labels	None
Team	None
Start date	None
Sprint	ITR2

Profile Settings

Description

"I should be able to change my profile settings"

These tasks corresponds with the following detailed user stories:

Detailed User Story	Description
(1.3) Manage 'My Courses' List	Add, remove, or update specific current course enrollments to filter matching algorithms.
(1.5) Edit study Preferences	Interface to customize "Study Vibe" tags such as "Quiet Focus" or "Group Discussion".
(4.1) Information & Privacy	Explicit settings to hide or reveal specific student information.

In Progress

Details

Assignee	SL Sean Lee-Wah
Priority	= Medium
Parent	None
Due date	None
Labels	None
Team	None
Start date	None
Sprint	ITR2

Connection & Status

Description

"I should be able to change my profile settings"

These tasks corresponds with the following detailed user stories:

Detailed User Story	Description
(1.6) View Network List	A page displaying all accepted connections and their current online status
(3.8) Online Status	View whether study group members are online, idle, or offline

Related work

In Progress

Details

Assignee	OF Omar Fakousa Assign to me
Priority	= Medium
Parent	None
Due date	None
Labels	None
Team	None
Start date	None
Sprint	ITR2

Map & Navigation

Description

"I should be able to see a map of the campus and see sessions involved with them"

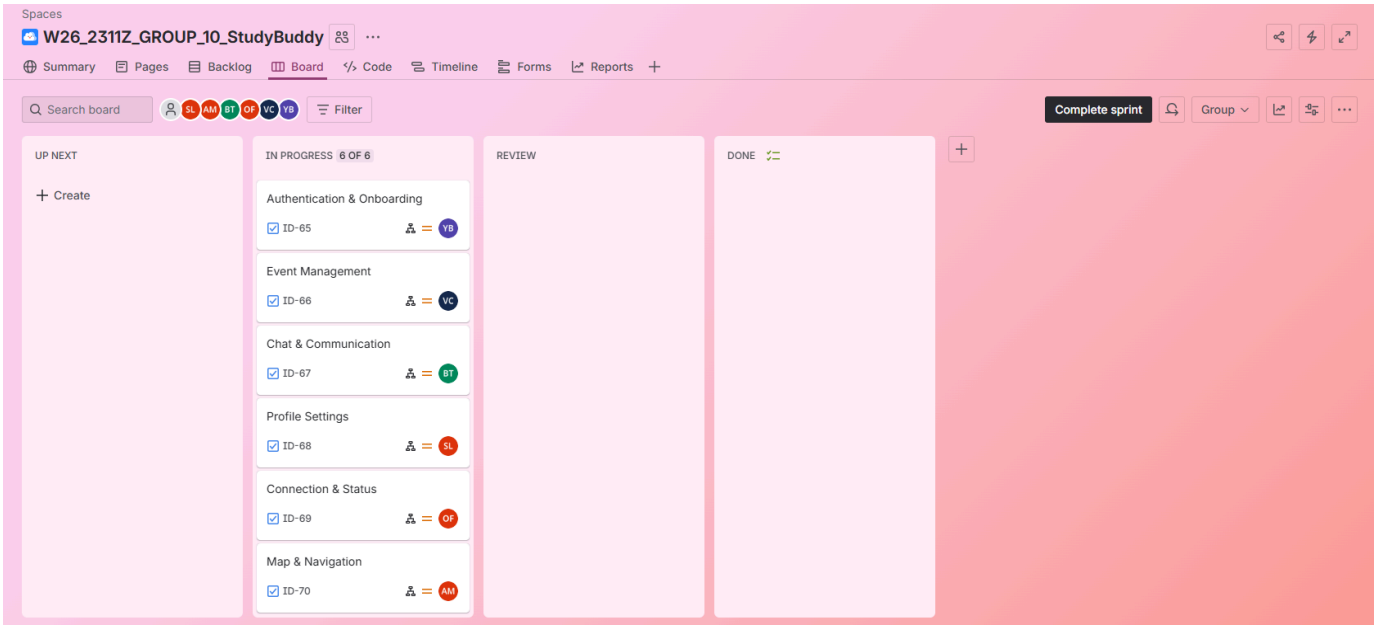
These tasks corresponds with the following detailed user stories:

Detailed User Story	Description
(3.1) View Possible Sessions	View available study sessions on a campus map based on current location.
(3.2) View Details of Possible Sessions	Access details like course, time, and duration before deciding to join.
(3.6) Know Current Location	Display the user's current location on the map.
(3.7, Optional) Navigation to Study Location	Directional navigation from the user's location to the chosen study spot.

In Progress

Details

Assignee	AM Aqsa Malik Assign to me
Priority	= Medium
Parent	None
Due date	None
Labels	None
Team	None
Start date	None
Sprint	ITR2



Development Tasks per User Story (Estimated/Actual Time):

(1.1) Create the base profile including name, email, and major

Task	Estimated Time	Actual Time
Display profile fields	4 hours	3 hours
Style the profile page	3 hours	3 hours

(4.4) Domain Filter: Restricting sign-ups to only @my.yorku.ca or @yorku.ca emails

Task	Estimated Time	Actual Time
Define allowed email domains.	2 hours	1 hour
Show error message for invalid emails.	1 hour	2 hours
Implement backend check to block unauthorized domains.	2 hours	2 hours

(4.2) Email authentication to ensure secure logins

Task	Estimated Time	Actual Time
Add email verification step during account sign-up.	3 hours	4 hours
Send verification link or code to the user's email.	2 hours	3 hours
Validate the link/code before activating the account.	2 hours	2 hours
Prevent login until the email is verified.	2 hours	3 hours

(2.1, 2.2) I should be able to select a place to create an event and be the host of it

Task	Estimated Time	Actual Time
Create event creation form with location selection.	2 hours	3 hours
Save event details and assign the creator as host.	3 hours	4 hours
Store event and host information in the database.	2 hours	4 hours
Display the hosted event in the user's events list.	3 hours	2 hours

(2.4) I should be able to join an existing and upcoming events

Task	Estimated Time	Actual Time
Display a list of available upcoming events.	2 hours	3 hours

Add a “Join Event” button for users.	2 hours	3 hours
Update the database to add the user to the event’s participants.	3 hours	5 hours
Show joined events in the user’s event list.	2 hours	4 hours

(2.3) Cancel Events: Allow the host to cancel an event they created

Task	Estimated Time	Actual Time
Add a “Cancel Event” option for the event host.	3 hours	4 hours
Verify that only the host can cancel the event.	4 hours	5 hours
Update the event status to “cancelled” in the database.	3 hours	2 hours

(2.5) Allow the user to cancel their own attendance to an event they joined

Task	Estimated Time	Actual Time
Add a “Leave Event” option for participants.	3 hours	4 hours
Verify the user is a participant of the event.	4 hours	4 hours
Remove the user from the event’s participant list in the database.	3 hours	2 hours
Update the user’s event list after leaving.	2 hours	1 hour

(1.4) I should be able to send friend requests to students after successfully completing a session

Task	Estimated Time	Actual Time
Detect when a session between students is completed.	3 hours	3 hours
Enable a “Send Friend Request” option for participants.	3 hours	5 hours
Create and store the friend request in the database.	2 hours	3 hours

(3.3) I should be able to use a chat feature to discuss study plans with group members

Task	Estimated Time	Actual Time
Create a group chat interface for event members.	4 hours	6 hours
Allow users to send and receive messages.	4 hours	5 hours
Store chat messages in the database.	1 hour	2 hours
Display messages to all members in the event chat.	2 hours	3 hours

(3.4) I should be able to attach files and links within the group chat

Task	Estimated Time	Actual Time
Add option to attach files or paste links in chat.	4 hours	5 hours
Upload and store shared files securely.	2 hours	3 hours
Display shared files and links in the chat messages.	3 hours	4 hours
Ensure only group members can access the attachments.	2 hours	3 hours

(3.9) Indicator fo Message: Viewing an indicator when someone is actively typing in the chat

Task	Estimated Time	Actual Time
Detect when a user starts typing in the chat.	3 hours	4 hours
Send typing status to other chat members.	2 hours	3 hours
Display a “typing…” indicator in the chat.	2 hour	2 hours
Remove the indicator when the user stops typing or sends the message.	2 hours	2 hours

(1.3) Add, remove, or update currently enrolled courses

Task	Estimated Time	Actual Time
Create interface to add, edit, or remove courses.	5 hours	7 hours
Validate course information before saving.	3 hours	2 hours
Update course data in the database.	2 hours	3 hours
Display the updated course list on the user profile.	2 hours	2 hours

(1.5) Customize “Study Vibe” and behavioral preferences

Task	Estimated Time	Actual Time
Create settings interface for study vibe preferences.	4 hours	5 hours
Allow users to select or update preference options.	3 hours	3 hours
Save preferences to the user profile in the database.	3 hours	2 hours
Use preferences when matching or forming study groups.	2 hours	4 hours

(4.1) Use explicit settings to reveal or hide certain information about the student

Task	Estimated Time	Actual Time
Create privacy settings for profile information.	4 hours	5 hours
Allow users to toggle visibility on/off.	3 hours	2 hours
Store visibility preferences in the database.	2 hours	2 hours
Display or hide profile information based on settings.	3 hours	4 hours

(1.6) Accessing a specific page that displays a list of all accepted connections

Task	Estimated Time	Actual Time
Create a page to display accepted connections.	6 hours	7 hours
Retrieve the user's confirmed connections from the database.	2 hours	2 hours
Display connections with basic profile information.	2 hours	4 hours

(3.8) View whether study group members are online, idle, or offline

Task	Estimated Time	Actual Time
Track user activity to determine status (online, idle, offline).	2 hours	3 hours
Update user status in real time.	2 hours	1 hour
Display status indicators next to group members.	2 hours	1 hour
Refresh status dynamically in the group interface.	2 hours	2 hours

(3.1) View available study sessions on campus map near your location

Task	Estimated Time	Actual Time
Integrate a campus map into the application.	4 hours	5 hours
Retrieve and display study session locations on the map.	2 hours	3 hours
Filter sessions based on the user’s location.	3 hours	3 hours
Allow users to select a session marker to view details.	3 hours	4 hours

(3.2) Check session details (course, time, duration) from the map before deciding to join

Task	Estimated Time	Actual Time
Display session information when a map marker is selected.	3 hours	4 hours
Show details such as course, time, and duration.	2 hours	3 hours
Provide an option to join the session from the details view.	2 hours	2 hours
Retrieve session data from the database.	3 hours	2 hours

(3.6) Display your current location on the map

Task	Estimated Time	Actual Time
Request permission to access the user's location.	2 hours	1 hour
Retrieve the user's current coordinates.	2 hours	2 hours
Display the location marker on the campus map.	2 hours	2 hours
Update the marker if the user's location changes.	2 hours	3 hours

(3.7) Get directional navigation from your location to the chosen study spot

Task	Estimated Time	Actual Time
Generate directions from current location to the spot.	2 hours	2 hours
Display the route visually on the pop-out map.	2 hours	3 hours

---

AI Usage Declaration:

Prompt: Based on this class code that I have provided, could you generate some thorough and useful test cases that cover all possible scenarios that could occur?

Response: The response was a thorough set of unit tests that covered the possible cases based on that specific class. For example, I used it for the StudentController.java class, and the response contained many test case scenarios that corresponded to profile details like entering student bio, courses, study vibes, etc.

Rationale: These test cases were very helpful because they allowed me to cover a variety of different test cases that could occur from this class, which may have not been as thorough if I were to create them manually.

- **Prompt: Would it be a better approach to integrate firebase authentication into the frontend or the backend?**

Response: The AI said it is better to integrate Firebase Authentication in the frontend and use the backend to trust and process the authenticated user’s identity, rather than handling the full login flow in the backend.

Rationale: While this approach was completely valid, our team decided it would be best to integrate the firebase authentication into the backend as it would work better with the workflow we already established from the previous iteration,

- Prompt: How can I implement a Spring Boot service that retrieves accepted connections from a Firestore collection where the logged-in user's UID may appear as either userA or userB?

Response: The AI recommended querying the connections collection for documents with status = "accepted", then looping through the results in Java. For each document, it would

compare the logged-in user's UID with userA and userB, and whichever one does not match would be treated as the connected user.

Rationale: We thought this was a good approach and we implemented this logic in our development.

- What is a good way to implement user presence in a Spring Boot backend using Firestore timestamps and determine whether a user is online, idle, or offline?

Response: The AI recommended storing a lastActiveAt timestamp in a presence collection and comparing it against the current server time. It would define thresholds such as within 2 minutes for online, within 10 minutes for idle, and anything older as offline.

Rationale: We thought this was a good approach although the exact timing for when users were last active was changed during development.

Prompt: Can you explain the architecture that we currently have in our wiki page? I've attached the architecture below. How should I update the architecture based on my features updates for the chat?

Response: Explanation of the architecture and what the differences are in chat features.

Rationale: The response provided a good architecture overview for iteration 2, based on that and its recommendations for what updates to make in the wiki page, I took its suggestions into account when deciding how to update the wiki page with chat feature updates.

Prompt:Could you help me generate a suite of test cases for the various classes I have and explain your rationale for it.

Response: The response was the creation of various test cases pertaining to the test files I had relating to the chat functionality.

Rationale: These test cases were a good starting point from which I took over and iterated on it to produce and write more comprehensive unit and integration tests for the entire project and its various classes.

Prompt: Unfortunately, our Firebase limit ran out of reads for the day…Thankfully, that won't matter 'cause we're working with Stub databases. Currently Firebase server access is hardcoded in the service files – this needs to be modified so that I can switch between Firebase and Stub database via applications.properties file. I believe the Student related java class files are related.

Response: It provided a plan, including what additional files to be added and noting that the student classes with an interface class to extend upon via a service (firebase connector) and repository (stub connector) class.

Rationale: For the sake of streamlining development, ensuring that each code change made sense (i.e no copy-pasting of code until determined what the change was and how it might affect other code).

Prompt: The chat branch is huge and I need a way to integrate it in the final ITR2 branch…I believe a lot of the legwork has been done by the team member however for integrating so, please direct me to the specific endpoints that I should consider

Response: It pinpointed the specific class, methods, and frontend code to consider when integrating the chat.

Rationale: For the sake of streamlining development, ensuring that each code change made sense (i.e no copy-pasting of code until determined what the change was and how it might affect other code), especially with a large merge.

Prompt: Before I work on connecting the chat, I think it is best to first work on being able to connect people. This way, what initiates the conversation is whether a user is connected or not. Help me integrate a group member's branch to get connections working

Response: The AI provided guidance on specific code changes required, including updating frontend to show actual connections, and the backend to store actual connections in the Firestore

Rationale: For the sake of streamlining development, ensuring that each code change made sense (i.e no copy-pasting of code until determined what the change was and how it might affect other code).

Prompt: I need to integrate the Firebase authentication into our application, of which filters out non YorkU emails and sends verification emails . How will this be wired to our Frontend?

Response: The AI provided guidance and clarified that the backend was (at the time) connected to Firebase so, when users enter a username and password, it gets sent to the backend, which forwards it to Firebase for verification. If successful, the backend will then forward the JWT token to the frontend, which then will put this in its header for each request it makes.

Rationale: The team did not have much familiarity with using firebase authentication, and hence, to speed up understanding and thus development of registration and sign-on, as well as how to show information in the frontend tied to the user who signed on with it.

Prompt (Figma AI): I'm not too picky with elements as long as the features here are present in the pure frontend: <information about the project here>. You should only

make the react components and layouts. I will then go through the code myself and make the firebase and springboot connections myself. I will also include the logic for going into screens myself. For data, just have placeholders for now

Response: An interactable React layout with mock data that can be navigated through.

Rationale: The team is inexperienced with using React for Frontend development and it would yield low return on time invested in learning React instead of working solely on the backend. As a result, the frontend was generated and manually reviewed for reference. Since we have a login system which should prevent users from abusing the system (i.e impersonating a user as another user), the application assumes the frontend is not secure.

Next Steps:

1. Assign and finish remaining user stories for ITR3 to ensure proper functionality of the app
2. Make final changes before delivery 2 to ensure no bugs or errors occur
3. Create additional unit tests to ensure proper functionality and code coverage.